Varying Slopes Models

General Principles

To model the relationship between predictor variables and a dependent variable while allowing for varying effects across groups or clusters, we use a *varying slopes* model.

This approach is useful when we expect the relationship between predictors and the dependent variable to differ across groups (e.g., different slopes for different subjects, locations, or time periods). This allows every unit in the data to have its own unique response to any treatment, exposure, or event, while also improving estimates via pooling.

Considerations

> **Note**
>
> - We have the same considerations as for Varying intercepts.
> - The idea is pretty similar to categorical models, where a slope is specified for each category. However, here, we also estimate relationships between different groups. This leads to a different mathematical approach, as to model these relationships between groups, we model a matrix of covariance .
> - To construct the covariance matrix, we use an SRS decomposition where S is a diagonal matrix of standard deviations and R is a correlation matrix. To model the correlation matrix, we use an $LKJcorr$ distribution parametrized with a single control parameter η that controls the amount of regularization. η is usually set to 2 to define a weakly informative prior that is skeptical of extreme correlations near -1 or 1. When we use $LKJcorr(1)$, the prior is flat over all valid correlation matrices. When the value is greater than 1, then extreme correlations are less likely.
> - The standard deviations in S are model with a prior that constrains them to strictly positive values.

Example

Below is an example code snippet demonstrating Bayesian regression with varying effects. This example is based on McElreath (2018).

Simulated data

Python (Raw)

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device-----------------------------------------------
m = bf(platform="cpu")

# Import Data & Data Manipulation ----------------------------------------------
# Import
data_path = m.load.sim_multivariate_normal(only_path=True)
m.data(data_path, sep=",")
m.data_on_model = dict(
    cafe=jnp.array(m.df.cafe.values, dtype=jnp.int32),
    wait=jnp.array(m.df.wait.values, dtype=jnp.float32),
    N_cafes=len(m.df.cafe.unique()),
    afternoon=jnp.array(m.df.afternoon.values, dtype=jnp.float32),
)


# Define model ----------------------------------------------
def model(cafe, wait, N_cafes, afternoon):
    a = m.dist.normal(5, 2, name="a")
    b = m.dist.normal(-1, 0.5, name="b")
    sigma_cafe = m.dist.exponential(1, shape=(2,), name="sigma_cafe")
    sigma = m.dist.exponential(1, name="sigma")
    Rho = m.dist.lkj(2, 2, name="Rho")
    cov = jnp.outer(sigma_cafe, sigma_cafe) * Rho
    a_cafe_b_cafe = m.dist.multivariate_normal(
        jnp.stack([a, b]), cov, shape=[N_cafes], name="a_b_cafe"
    )

    a_cafe, b_cafe = a_cafe_b_cafe[:, 0], a_cafe_b_cafe[:, 1]
    mu = a_cafe[cafe] + b_cafe[cafe] * afternoon
```

```
    m.dist.normal(mu, sigma, obs=wait)


# Run sampler ----------------------------------------------------
m.fit(model, progress_bar=False)

# Summary --------------------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
Rho[0, 0]	1.00	0.00	1.00	1.00	4000.00	4000.00	NaN	0.00	NaN
Rho[0, 1]	-0.57	0.21	-0.85	-0.18	1833.18	1493.86	1.00	0.00	0.00
Rho[1, 0]	-0.57	0.21	-0.85	-0.18	1833.18	1493.86	1.00	0.00	0.00
Rho[1, 1]	1.00	0.00	1.00	1.00	4014.13	3913.63	1.00	0.00	0.00
a	3.86	0.27	3.45	4.29	3168.73	2526.20	1.00	0.00	0.01
a_b_cafe[0, 0]	4.23	0.21	3.89	4.57	3831.89	2916.19	1.00	0.00	0.00
a_b_cafe[0, 1]	-1.49	0.24	-1.89	-1.13	3130.28	2793.19	1.00	0.00	0.00
a_b_cafe[1, 0]	0.39	0.23	0.02	0.76	1907.67	1616.57	1.00	0.01	0.00
a_b_cafe[1, 1]	-0.71	0.30	-1.18	-0.23	1279.76	1009.81	1.00	0.01	0.01
a_b_cafe[2, 0]	4.43	0.21	4.10	4.76	3536.65	2937.85	1.00	0.00	0.00
a_b_cafe[2, 1]	-1.40	0.23	-1.77	-1.04	3426.00	2987.08	1.00	0.00	0.00
a_b_cafe[3, 0]	3.64	0.22	3.30	4.00	2810.12	2747.52	1.00	0.00	0.00
a_b_cafe[3, 1]	-1.60	0.26	-2.04	-1.23	1741.94	2284.24	1.00	0.01	0.00
a_b_cafe[4, 0]	3.20	0.22	2.85	3.55	2421.35	1581.28	1.00	0.00	0.00
a_b_cafe[4, 1]	-1.02	0.25	-1.39	-0.60	1609.16	1690.18	1.00	0.01	0.00
a_b_cafe[5, 0]	3.01	0.21	2.67	3.36	2825.27	3012.78	1.00	0.00	0.00
a_b_cafe[5, 1]	-1.40	0.25	-1.81	-1.03	2173.10	2613.80	1.00	0.01	0.00
a_b_cafe[6, 0]	2.88	0.23	2.50	3.24	2378.17	2573.91	1.00	0.00	0.00
a_b_cafe[6, 1]	-0.87	0.28	-1.31	-0.41	1099.63	920.22	1.00	0.01	0.00
a_b_cafe[7, 0]	3.02	0.21	2.69	3.36	3656.27	3029.73	1.00	0.00	0.00
a_b_cafe[7, 1]	-1.43	0.25	-1.84	-1.06	2446.92	2275.36	1.00	0.01	0.00
a_b_cafe[8, 0]	5.48	0.22	5.15	5.82	3082.65	2908.80	1.00	0.00	0.00
a_b_cafe[8, 1]	-1.50	0.24	-1.88	-1.10	2572.46	2912.21	1.00	0.00	0.00
a_b_cafe[9, 0]	3.45	0.21	3.12	3.78	3841.15	3076.79	1.00	0.00	0.00
a_b_cafe[9, 1]	-1.36	0.23	-1.73	-1.01	3363.83	2778.71	1.00	0.00	0.00

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a_b_cafe[10, 0]	3.96	0.21	3.63	4.30	3633.44	3216.91	1.00	0.00	0.00
a_b_cafe[10, 1]	-1.12	0.25	-1.48	-0.70	1827.74	1647.77	1.00	0.01	0.00
a_b_cafe[11, 0]	4.19	0.21	3.85	4.52	3108.86	2767.31	1.00	0.00	0.00
a_b_cafe[11, 1]	-1.30	0.23	-1.65	-0.92	2701.40	2786.29	1.00	0.00	0.00
a_b_cafe[12, 0]	5.38	0.21	5.04	5.73	1962.84	1501.48	1.00	0.00	0.00
a_b_cafe[12, 1]	-1.66	0.24	-2.06	-1.29	2600.85	2845.69	1.00	0.00	0.00
a_b_cafe[13, 0]	3.53	0.21	3.19	3.86	3590.51	2929.39	1.00	0.00	0.00
a_b_cafe[13, 1]	-1.21	0.23	-1.56	-0.82	3109.55	2329.46	1.00	0.00	0.00
a_b_cafe[14, 0]	3.48	0.21	3.15	3.82	3171.12	3149.05	1.00	0.00	0.00
a_b_cafe[14, 1]	-1.15	0.24	-1.51	-0.76	2509.79	2642.58	1.00	0.00	0.00
a_b_cafe[15, 0]	4.30	0.21	3.95	4.63	4206.33	3108.57	1.00	0.00	0.00
a_b_cafe[15, 1]	-1.40	0.23	-1.77	-1.04	3794.18	2676.59	1.00	0.00	0.00
a_b_cafe[16, 0]	4.55	0.21	4.21	4.90	3501.06	3148.59	1.00	0.00	0.00
a_b_cafe[16, 1]	-1.60	0.24	-2.00	-1.24	2568.49	2413.74	1.00	0.00	0.00
a_b_cafe[17, 0]	4.67	0.21	4.33	5.01	3707.24	2557.49	1.00	0.00	0.00
a_b_cafe[17, 1]	-1.36	0.23	-1.72	-0.99	2810.02	2862.34	1.00	0.00	0.00
a_b_cafe[18, 0]	5.42	0.22	5.07	5.78	3355.84	2427.22	1.00	0.00	0.00
a_b_cafe[18, 1]	-1.76	0.26	-2.20	-1.37	1983.42	2629.06	1.00	0.01	0.00
a_b_cafe[19, 0]	3.90	0.21	3.56	4.23	3156.97	2959.98	1.00	0.00	0.00
a_b_cafe[19, 1]	-1.21	0.23	-1.56	-0.83	2608.29	2865.61	1.00	0.00	0.00
b	-1.32	0.11	-1.49	-1.14	1850.78	2104.88	1.00	0.00	0.00
sigma	0.54	0.03	0.50	0.59	3231.11	2846.60	1.00	0.00	0.00
sigma_cafe[0]	1.18	0.20	0.90	1.52	4006.71	2682.78	1.00	0.00	0.00
sigma_cafe[1]	0.36	0.12	0.18	0.56	571.47	572.62	1.01	0.00	0.00

Python (Build in function)

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device-----------------------------------------------
m = bf(platform="cpu")

# Import Data & Data Manipulation ----------------------------------------------

data_path = m.load.sim_multivariate_normal(only_path=True)
m.data(data_path, sep=",")
m.data_on_model = dict(
    cafe=jnp.array(m.df.cafe.values, dtype=jnp.int32),
```

```
    wait=jnp.array(m.df.wait.values, dtype=jnp.float32),
    N_cafes=len(m.df.cafe.unique()),
    afternoon=jnp.array(m.df.afternoon.values, dtype=jnp.float32),
)


# Define model ------------------------------------------------
def model(cafe, wait, N_cafes, afternoon):

    sigma = m.dist.exponential(1, name="sigma")

    varying_intercept, varying_slope = m.effects.varying_effects(
        N_vars=1, N_group=N_cafes, group_id=cafe, group_name="cafe", centered=False
    )

    mu = varying_intercept + varying_slope * afternoon
    m.dist.normal(mu, sigma, obs=wait)


# Run sampler -------------------------------------------------
m.fit(model, progress_bar=False)
```

```
jax.local_device_count 16
```

R

```
library(BayesForge)
jnp = reticulate::import('jax.numpy')

# Setup platform----------------------------------------------------
m=importBF(platform='cpu')

# Import data -------------------------------------------------
data_path = m$load$sim_multivariate_normal(only_path = TRUE)
m$data(data_path, sep = ',')
m$data_on_model <- list(
  cafe = jnp$array(as.integer(m$df$cafe), dtype = jnp$int32),
  wait = jnp$array(as.numeric(m$df$wait), dtype = jnp$float32),
  N_cafes = as.integer(length(unique(m$df$cafe))),
  afternoon = jnp$array(as.numeric(m$df$afternoon), dtype = jnp$float32)
```

```
)

# Define model ------------------------------------------------
model <- function(cafe, wait, N_cafes, afternoon){
  sigma = m$dist$exponential(1, name = 'sigma')

  veff = m$effects$varying_effects(
    N_vars = 1L, N_group = N_cafes, group_id = cafe, group_name = 'cafe', centered = FALSE
  )
  varying_intercept = veff[[1]]
  varying_slope = veff[[2]]

  mu = varying_intercept + varying_slope * afternoon

  m$dist$normal(mu, sigma, obs = wait, name = 'wait')
}

# Run MCMC ----------------------------------------------------
m$fit(model, progress_bar = FALSE) # Optimize model parameters through MCMC sampling

# Summary -----------------------------------------------------
m$summary() # Get posterior distribution
```

Julia

```
using PyBayesForge

# Setup device------------------------------------------------
m = importBF(platform="cpu")

# Import Data & Data Manipulation -----------------------------------------------
# Import
data_path = m.load.sim_multivariate_normal(only_path = true)

m.data(data_path, sep=',')
m.data_on_model = pybuiltins.dict(
    cafe = jnp.array(m.df.cafe.values, dtype=jnp.int32),
    wait = jnp.array(m.df.wait.values, dtype=jnp.float32),
    N_cafes = length(m.df.cafe.unique()),
    afternoon = jnp.array(m.df.afternoon.values, dtype=jnp.float32)
)
```

```
# Define model ----------------------------------------------
@BF function model(cafe, wait, N_cafes, afternoon)
    a = m.dist.normal(5, 2,  name = "a")
    b = m.dist.normal(-1, 0.5, name = "b")
    sigma_cafe = m.dist.exponential(1, shape=(2,),  name = "sigma_cafe")
    sigma = m.dist.exponential( 1,  name = "sigma")
    Rho = m.dist.lkj(2, 2, name = "Rho")
    cov = jnp.outer(sigma_cafe, sigma_cafe) * Rho
    a_cafe_b_cafe = m.dist.multivariate_normal(jnp.stack([a, b]), cov, shape = [N_cafes], nam

    a_cafe, b_cafe = a_cafe_b_cafe[:, 0], a_cafe_b_cafe[:, 1]
    mu = a_cafe[cafe] + b_cafe[cafe] * afternoon
    m.dist.normal(mu, sigma, obs=wait)

end

# Run mcmc ---------------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary ----------------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

Centered parameterization

The Gaussian Mixture Model is a hierarchical model where each data point is generated from one of K distinct multivariate Gaussian distributions.

The varying intercepts (α_k) and slopes (β_k) are modeled using a *Multivariate Normal distribution*:

$$\begin{pmatrix} \alpha_k \\ \beta_k \end{pmatrix} \sim \text{MultivariateNormal}\left(\begin{pmatrix} \bar{\alpha} \\ \bar{\beta} \end{pmatrix}, \text{diag}(\varsigma)\ \Omega\ \text{diag}(\varsigma) \right)$$

$$\bar{\alpha} \sim \text{Normal}(0, 1)$$

$$\bar{\beta} \sim \text{Normal}(0, 1)$$

$$\varsigma \sim \text{Exponential}(1)$$

$$\Omega \sim \text{LKJ}(\eta)$$

Where:

- $\begin{pmatrix} \bar{\alpha} \\ \bar{\beta} \end{pmatrix}$ is a vector composed from concatenating a parameter for the global intercept and a parameter vector of the global slopes.
- ς is a vector giving the standard deviation of the random effects for the intercept and slopes across groups.
- Ω is the correlation matrix.

Non-centered parameterization

For computational reasons, it is often better to implement a non-centered parameterization that is equivalent to the *Multivariate Normal distribution* approach:

$$\begin{pmatrix} \alpha_k \\ \beta_k \end{pmatrix} = \begin{pmatrix} \bar{\alpha} \\ \bar{\beta} \end{pmatrix} + \varsigma \circ \left(L \cdot \begin{pmatrix} \hat{\alpha}_k \\ \hat{\beta}_k \end{pmatrix} \right)$$

$$\bar{\alpha} \sim \text{Normal}(0, 1)$$

$$\bar{\beta} \sim \text{Normal}(0, 1)$$

$$\varsigma \sim \text{Exponential}(1)$$

$$L \sim \text{LKJ Cholesky}(\eta)$$

$$\hat{\alpha}_k \sim \text{Exponential}(1)$$

$$\hat{\beta}_k \sim \text{Exponential}(1)$$

- Where:
  - $\sigma_\alpha \sim \text{Exponential}(1)$ is the prior standard deviation among intercepts.
  - $\sigma_\beta \sim \text{Exponential}(1)$ is the prior standard deviation among slopes.
  - $L \sim \text{LKJcorr}(\eta)$ is the a cholesky factor of the correlation matrix matrix using the Cholesky Factor

Multivariate Model with One Random Slope for Each Variable

We can apply a multivariate model similarly to Chapter 2. In this case, we apply the same principle, but with a covariance matrix with a dimension equal to the number of varying slopes we define. For example, if we want to generate random slopes for i observations in a model with two independent variables X_1 and X_2, we can define the formula as follows:

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_i + \beta_{k(i)} X_{1i} + \gamma_{k(i)} X_{2i}$$

$$\begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} \sim \begin{pmatrix} \bar{\alpha} \\ \bar{\beta} \\ \bar{\gamma} \end{pmatrix} + \varsigma \circ \left(L \cdot \begin{pmatrix} \hat{\alpha}_k \\ \hat{\beta}_k \\ \hat{\gamma}_k \end{pmatrix} \right)$$

$$\bar{\alpha} \sim \text{Normal}(0, 1)$$
$$\bar{\beta} \sim \text{Normal}(0, 1)$$

$$\bar{\gamma} \sim \text{Normal}(0, 1)$$

$$\varsigma \sim \text{Exponential}(1)$$
$$L \sim \text{LKJ Cholesky}(2)$$

$$\hat{\alpha}_k \sim \text{Exponential}(1)$$

$$\hat{\beta}_k \sim \text{Exponential}(1)$$

$$\hat{\gamma}_k \sim \text{Exponential}(1)$$

Model validation tests

- Comparison with Stan: `12.Varying effects.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence).
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> - We can apply interaction terms similarly to Chapter 3.
> - We can apply categorical variables similarly to Chapter 4.
> - We can apply multiple random effects in BayesForge as follows:
>
> **Python**

```
def model(cafe, wait, N_cafes, afternoon, state):
    sigma = m.dist.exponential( 1,  name = 'sigma')

    #  Note: `alpha_bar` and `beta_bar` are set to zero to ensure the same intercepts for
    varying_intercept_1, varying_slope_1 = m.effects.varying_effects(
        N_vars = 1,
        N_group = N_cafes,
        group_id = cafe,
        group_name='cafe',
        alpha_bar = jnp.array([0]),
        beta_bar = jnp.array([0]),
        centered=True,
    )

    varying_intercept_2, varying_slope_2 = m.effects.varying_effects(
        N_vars = 1,
        N_group = 4,
        group_id = state,
        group_name='states',
        alpha_bar = jnp.array([0]),
        beta_bar = jnp.array([0]),
        centered=True,
    )

    varying_intercept = varying_intercept_1 + varying_intercept_2
    varying_slope = varying_slope_1 + varying_slope_2

    mu = varying_intercept + varying_slope * afternoon
    m.dist.normal(mu, sigma, obs=wait)
```

R

```
model <- function(cafe, wait, N_cafes, afternoon, state) {
  sigma <- m$dist$exponential(1, name = "sigma")

  #  Note: `alpha_bar` and `beta_bar` are set to zero to ensure the same intercepts for al
  veff_1 <- m$effects$varying_effects(
    N_vars = 1L,
    N_group = N_cafes,
    group_id = cafe,
    group_name = "cafe",
    alpha_bar = jnp$array(list(0)),
    beta_bar = jnp$array(list(0)),
    centered = TRUE
  )

  veff_2 <- m$effects$varying_effects(
    N_vars = 1L,
    N_group = 4L,
    group_id = state,
    group_name = "states",
    alpha_bar = jnp$array(list(0)),
    beta_bar = jnp$array(list(0)),
    centered = TRUE
  )

  varying_intercept <- veff_1[[1]] + veff_2[[1]]
  varying_slope <- veff_1[[2]] + veff_2[[2]]

  mu <- varying_intercept + varying_slope * afternoon
  m$dist$normal(mu, sigma, obs = wait)
}
```

Julia

```
@BF function model(cafe, wait, N_cafes, afternoon, state)
    sigma = m.dist.exponential(1, name="sigma")

    #  Note: `alpha_bar` and `beta_bar` are set to zero to ensure the same intercepts for
    varying_intercept_1, varying_slope_1 = m.effects.varying_effects(
        N_vars=1,
        N_group=N_cafes,
        group_id=cafe,
        group_name="cafe",
        alpha_bar=jnp.array([0]),
        beta_bar=jnp.array([0]),
        centered=true,
    )

    varying_intercept_2, varying_slope_2 = m.effects.varying_effects(
        N_vars=1,
        N_group=4,
        group_id=state,
        group_name="states",
        alpha_bar=jnp.array([0]),
        beta_bar=jnp.array([0]),
        centered=true,
    )

    varying_intercept = varying_intercept_1 + varying_intercept_2
    varying_slope = varying_slope_1 + varying_slope_2

    mu = varying_intercept + varying_slope * afternoon
    m.dist.normal(mu, sigma, obs=wait)
end
```

Where `state` is a second varying effect, representing cafes clustered by state.

Note: `alpha_bar` and `beta_bar` are set to zero to ensure the same intercepts for all varying effects.

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan.* Chapman; Hall/CRC.